Z.AI · DESKTOP AUTOMATION

Z.AGENT

Autonomous Desktop Agent

Control your computer remotely via Telegram.

10 LLM providers · 88 actions · 16 modules · 26 core components

Version 4.0 · June 2026

Powered by GLM-4.6, GPT-4o, Claude 3.5, Mistral Large, Llama 3.1, and more

Table of Contents

1. Introduction and Overview

1.1 What is Z.AGENT?

Z.AGENT is an autonomous desktop agent powered by 10 major LLM providers. It transforms your computer into a remotely-controllable system: from your smartphone via Telegram, or from a local web dashboard, you assign tasks in natural language and it executes them automatically. The agent can move the cursor, click, type text, manage your files, read and send emails, check your calendar, drive a web browser, and control system processes. It operates in full autonomy, without requiring human supervision, making it ideal for scenarios like managing your workstation during absences, automating repetitive tasks, or executing requests remotely when you don't have access to your machine.

The project's philosophy is simple: transform any LLM into a true desktop agent. Rather than limiting AI to generating code or answering questions, Z.AGENT gives it physical control of your computer. You can be traveling, in a meeting, or simply away — the agent stays active, listens to your Telegram instructions, plans the necessary actions, and executes them using the right modules for each task type. The system relies on a multi-LLM architecture where you can use z.ai GLM models, OpenAI GPT, Anthropic Claude, Mistral, or any of 10 supported providers, with automatic fallback if one fails.

1.2 Key Use Cases

Automatic organization

The agent monitors your Downloads folder and sorts files by type (images, documents, archives, code, videos). It can also archive old files and clean up duplicates.

Email management

While you're away, the agent reads unread emails, sends you a summary via Telegram, flags urgent ones, and can auto-reply to simple messages with your approval.

Meeting preparation

The agent checks your calendar, prepares the day's agenda, opens relevant documents in the right applications, and sends reminders to participants.

Browser automation

The agent opens websites, fills forms, extracts data, and scrapes pages as needed. Ideal for monitoring prices, filling declarations, or collecting information.

System maintenance

The agent monitors running processes, kills those consuming too many resources, launches applications you need, and notifies you of problems.

Scheduled tasks

Combine the agent with the built-in scheduler to run recurring tasks: backups, cleanups, checks, or any other automatable scenario.

Document research

Add documents to the knowledge base (PDF, DOCX, TXT) and ask the agent to find information semantically — 'What does our security policy say about X?'

Voice control

Send voice messages to the Telegram bot — Whisper transcribes them and the agent processes them as text. Hands-free operation.

1.3 Supported LLM Providers

Z.AGENT supports 10 LLM providers through a unified interface with automatic fallback. Each provider can be configured independently, and the agent automatically falls back to the next provider if the primary one fails. This ensures maximum reliability and lets you choose the best model for each task type.

Provider	Default Model	Strengths
z.ai (default)	GLM-4.6	Best price/performance, vision support, coding plan SDK
OpenAI	GPT-4o	Most capable, vision, tool calling
Anthropic	Claude 3.5 Sonnet	Best for reasoning, long context
Mistral	Mistral Large	Open-weight, Codestral for code
NVIDIA NIM	Llama 3.1 405B	Free tier, large models
Groq	Llama 3.3 70B	Ultra-fast inference
DeepSeek	DeepSeek-V3	Best value, reasoning model
Ollama	Llama 3.2	Local, free, private
Together AI	Llama 3.3 70B	Open models at scale
Fireworks AI	Llama 3.3 70B	Fast inference, good pricing

2. Architecture

2.1 Overview

Z.AGENT's architecture is organized into clearly separated layers. At the top, user interfaces (Telegram, web dashboard, CLI, webhooks) receive requests and pass them to the agent. The agent orchestrates three specialized components: the planner that decomposes the task, the executor that runs the actions, and the memory that persists context. Actions are ultimately routed to functional modules (screen, files, email, calendar, browser, system, windows, code, web, voice, plugin, mcp, vision, knowledge base, slack) which interact with the host system.

2.2 Core Components

Agent Loop

The main loop that runs continuously in the background. It listens to the task queue, dequeues requests in order, and processes them one by one through the planning -> execution pipeline. Between tasks, it performs periodic housekeeping: screenshot cleanup, calendar reminder checks, scheduled task execution, and system monitoring. The agent exposes its state (idle, planning, executing, paused, stopped) which is visible in real time on the dashboard and notified via Telegram.

ReAct Loop

Instead of generating one plan and blindly executing it, the agent uses a ReAct (Reason + Act + Observe + Critique) loop. At each turn, it reasons about what to do next, takes one action, observes the result, critiques whether it worked, and decides the next step. This makes the agent adaptive — it recovers from failures, changes strategy mid-task, and stops early when the goal is achieved.

Multi-LLM Provider

A unified interface across 10 LLM providers (z.ai, OpenAI, Anthropic, Mistral, NVIDIA, Groq, DeepSeek, Ollama, Together, Fireworks). If the primary provider fails, it automatically falls back to the next configured provider. Per-role routing lets you use different providers for planning, vision, and execution.

Executor

Receives the plan and executes each step sequentially. It routes each action to the appropriate module handler, applies security checks before execution, and logs the result (success, error, duration) to the audit log. A configurable safety delay separates each action.

Perception (VLM)

Uses GLM-4V (or any vision-capable provider) to visually understand the screen. It captures screenshots on demand, resizes them for token optimization, and sends them to the vision model with a specific question. It can locate a UI element by description, describe the overall screen content, or verify that an action had the expected effect.

Vector Memory

Long-term semantic memory using embeddings. The agent can recall what it learned about any topic without knowing the exact key. Memories are boosted by importance, recency, and recall frequency. Uses cosine similarity for semantic

search.

Auto Skill Creator

Automatically detects recurring patterns in successful tasks. When a pattern appears 2+ times with 70%+ success rate, it automatically creates a reusable skill. This means the agent gets faster over time — it reuses learned skills instead of re-planning from scratch.

Cost Tracker

Tracks every API call's token usage and estimated cost (USD). Provides aggregations by period (today/week/month/all), by model, by role, and by task. Includes a trend chart and top tasks by cost.

Audit Log

An append-only security trail of every action executed by the agent. Records who triggered it, what action was attempted, when, whether it was allowed or blocked, and the result. Sensitive parameters are automatically redacted. Essential for security investigations and compliance.

2.3 Task Execution Flow

The complete flow of a task, from user instruction to final result, follows these steps:

Step	Component	Action
1	Interface	User sends a request via Telegram, dashboard, CLI, or webhook.
2	Agent Loop	Task is added to the queue with a unique ID.
3	ReAct Loop	GLM-4.6 (or configured provider) reasons about the next action to take.
4	Executor	The action is executed by the appropriate module after security checks.
5	Perception	If the action needs visual feedback, GLM-4V analyzes the screen to confirm.
6	ReAct Loop	Agent critiques the result and decides the next action (or declares goal achieved).
7	Memory	Task, result, activity, and patterns are recorded in memory systems.
8	Interface	Result is notified to the user (Telegram push + dashboard real-time).

3. Installation and Configuration

3.1 System Requirements

Z.AGENT runs on Windows 10/11, macOS 11+, and Linux (Ubuntu 20.04+ or equivalent). It requires Python 3.10 or higher, plus administrator rights for installing system dependencies. On Linux, you'll need X11 libraries for screen control. On macOS, you must grant Accessibility permissions to Python in System Preferences > Security & Privacy > Privacy > Accessibility. On Windows, screen control works natively without additional configuration. For voice control, ffmpeg must be installed on the system.

3.2 Installing Dependencies

Clone or download the project, then install Python dependencies:

```
cd z-agent-desktop python -m venv venv source venv/bin/activate # Linux/macOS # or: venv\Scripts\activate # Windows pip install -r requirements.txt playwright install chromium # On Windows (for full Windows control): pip install pywin32 # For voice control: # apt install ffmpeg # Linux # brew install ffmpeg # macOS
```

3.3 Getting API Keys

z.ai API Key (required)

Visit https://z.ai/ and create an account. Once logged in, access the developer dashboard and generate an API key. This key is required to call GLM models (4.6, 4V, 4.5, 5.1, 5.2). Keep it secure — it will be used for all AI requests. Cost is billed per usage but remains very affordable for personal use — approximately $0.01 per complex task on average.

Telegram Bot Token (recommended)

To control the agent remotely via Telegram, create a bot by talking to @BotFather on Telegram. Send /newbot, follow the instructions to name your bot, and retrieve the API token. Then get your Telegram user ID by talking to @userinfobot — this ID will be used to restrict access to your bot (no one else can give orders to your agent).

Additional LLM Providers (optional)

Z.AGENT supports 10 LLM providers. Set any of these environment variables to enable additional providers. The agent automatically detects which providers are available and adds them to the fallback chain:

```
# .env file ZAI_API_KEY=your-z.ai-key OPENAI_API_KEY=sk-... ANTHROPIC_API_KEY=sk-ant-... MISTRAL_API_KEY=... NVIDIA_API_KEY=nvapi-... GROQ_API_KEY=gsk_... DEEPSEEK_API_KEY=sk-... TOGETHER_API_KEY=... FIREWORKS_API_KEY=... # Ollama runs locally — no key needed, just install from https://ollama.com
```

3.4 Configuration File

All sensitive values should be placed in a .env file at the project root. The config/config.yaml file contains all other settings: watched folders, organization rules, AI models to use, VLM confidence thresholds, perception intervals, security policy, LLM provider routing, voice settings, vision streaming, MCP servers, and more. Each section is commented in detail.

3.5 Verification

Before first launch, run the verification command to ensure everything is in place:

```
python main.py --check
```

This command checks the config file, the existence of API keys, and the correct installation of each Python dependency. It displays a clear report with green checkmarks for validated items and orange warnings for missing items.

3.6 First Launch

Server mode (default) starts the agent, Telegram bot, FastAPI web API on port 8765, scheduler, and file watcher simultaneously:

```
python main.py
```

For quick testing without Telegram or dashboard, use the interactive CLI mode: you type requests directly in the terminal and see results. To run a single task and exit, use --task "your request". The CLI mode is ideal for debugging a specific module or validating that the agent understands your requests correctly.

4. Multi-LLM Provider (10 Providers)

4.1 Overview

Z.AGENT supports 10 LLM providers through a unified interface. This means you can use any combination of providers — z.ai as primary with OpenAI as fallback, or Claude for reasoning with GPT-4o for vision, or any other configuration you prefer. The agent automatically falls back to the next provider if the primary one fails (rate limit, network error, etc.).

4.2 Configuration

Configure the primary provider and fallback chain in config.yaml:

```
# config.yaml llm_provider: primary: zai # Primary provider fallbacks: [openai, anthropic] # Fallback chain routing: # Per-role routing (optional) planner: anthropic # Use Claude for planning vision: openai # Use GPT-4o for vision executor: groq # Use Groq for fast execution
```

4.3 Switching Providers from Dashboard

The dashboard includes an LLM Provider Switcher panel that lets you:

- See all 10 providers and their availability (green/red dot)
- Set the primary provider with one click
- Test each provider connection with a simple 'OK' request
- See response time for each test

4.4 Cost Comparison

Each provider has different pricing. The Cost Tracker panel shows you exactly how much you're spending on each provider, so you can optimize. For example, Groq is the cheapest for fast tasks, DeepSeek offers the best value for reasoning, and z.ai GLM models are very affordable for general use. Ollama is completely free (local inference) but requires a GPU for good performance.

5. Functional Modules

Z.AGENT is composed of 16 specialized modules, each responsible for a functional domain. Each module registers its actions with the central executor, which routes planner requests to the right handler. This modular architecture lets you add, remove, or replace a module without affecting the rest of the system.

Screen Control

Cursor, keyboard, windows — generic UI automation via PyAutoGUI + VLM. 10 actions: click_element, click_xy, type_text, press_key, hotkey, scroll, screenshot, wait, find_and_click, drag.

File Manager

Organize, move, copy, rename, delete (to trash by default), search by name or content, read/write text files. 10 actions including auto-organize by extension category.

Email Client

IMAP/SMTP — read, send, reply, search (Gmail, Outlook, Yahoo). Supports attachments, HTML, CC/BCC. 6 actions.

Calendar

ICS — list, create, delete events, search, set reminders. Supports recurring events. 5 actions.

Browser Control

Playwright — open URLs, click CSS selectors, fill forms, extract text/HTML, screenshot, scroll, evaluate JavaScript. 8 actions.

System Control

Launch/kill apps, list processes, system notifications, clipboard, open files, system info, run commands. 9 actions.

Windows Control

100% Windows desktop control: PowerShell, registry (read/write/delete), services (start/stop/list), window management (focus/close/minimize/maximize/move), volume, brightness, wallpaper, installed apps (list/uninstall), Wi-Fi, event log, COM automation (Outlook/Excel/Word), taskbar pin, environment variables. 25 actions.

Code Interpreter

Sandboxed Python execution with AST validation, import whitelist, resource limits (CPU/RAM), filesystem restrictions. 4 actions: run_python, evaluate, list_files, read_file.

Web Search

Real-time web search and page reading via z-ai-web-dev-sdk. Deep research mode reads top results and synthesizes findings. 4 actions: search, read_page, fetch, research.

Voice Control

Whisper STT (API or local) + TTS. Send voice messages to Telegram — the agent transcribes and processes them. 3 actions: transcribe, speak, list_voices.

Plugin Marketplace

Install third-party plugins from local paths, zips, or git URLs. Enable/disable/uninstall. Auto-installs plugin requirements. 7 actions.

MCP Client

Model Context Protocol — connect to external tool servers (filesystem, GitHub, Postgres, Slack, Puppeteer). Discover and call tools via JSON-RPC. 5 actions.

Vision Stream

Continuous screen monitoring with change detection (perceptual hash). Watchers trigger notifications when specific UI elements appear. 5 actions.

Knowledge Base

RAG with document chunking (1000 chars, 200 overlap), embeddings via z.ai API, cosine similarity search. Supports PDF, DOCX, TXT, CSV, JSON. 5 actions.

Slack Notifier

Send messages, files, list channels via official Slack SDK. 4 actions.

Custom Module Template

Documented template for creating your own modules. Follows the same register() pattern as built-in modules.

6. Agentic Core — ReAct Loop & Multi-Agent

6.1 ReAct Loop

Instead of generating one plan and executing it blindly, Z.AGENT uses a ReAct (Reason + Act + Observe + Critique) loop. At each turn, the agent: (1) reasons about what to do next, (2) takes one action, (3) observes the result, (4) critiques whether it worked, and (5) decides the next step. This makes the agent adaptive — it recovers from failures, changes strategy mid-task, asks for clarification when needed, and stops early when the goal is achieved. Maximum 50 turns per task (configurable).

6.2 Multi-Agent Orchestrator

For complex tasks, the orchestrator decomposes them into sub-tasks assigned to specialized sub-agents. Each sub-agent type has a specific role: researcher (gathers information), coder (writes and tests code), file_organizer (file management), communicator (emails, Slack, notifications), browser_agent (browser automation), system_agent (system administration). Sub-agents can run in parallel (asyncio.gather) with dependency management. Results are synthesized into a final answer by the orchestrator.

6.3 Skill Library & Auto Skill Creator

The agent learns from its successes. When a task succeeds, the action sequence is analyzed. If the same pattern appears 2+ times with 70%+ success rate, the Auto Skill Creator automatically saves it as a reusable skill. When a similar goal appears in the future, the agent can reuse the learned skill — saving planning time and tokens. Skills can also be created manually and imported/exported for sharing.

6.4 Native Tool Calling

Z.AGENT uses the native function-calling API of GLM and other providers. The LLM can call tools directly — we execute them, feed results back, and the LLM continues. This is more reliable than manual JSON parsing, uses fewer tokens, and supports multi-round tool calling (up to 5 rounds).

7. Memory Systems

7.1 Overview

Z.AGENT has 4 memory systems that work together to give the agent long-term context:

Vector Memory (Semantic)

Long-term semantic memory using embeddings. The agent can recall what it learned about any topic without knowing the exact key. Each memory has: text content, embedding vector, metadata (type, tags, source, timestamp), importance score (decays over time, boosted when recalled). Uses cosine similarity for semantic search. Boosted by importance + recency + recall frequency.

Conversation Context (Multi-task)

Per-session context that persists across tasks. Follow-up requests like 'do the same for Documents' work because the agent remembers past turns. Old turns are compacted into summaries to fit in context. Sessions can be listed, switched, and searched.

Skill Library (Action Sequences)

Saved action sequences reusable across tasks. Each skill has: name, description, goal, action sequence, tags, language, use count, success count. Searchable by text query. Skills are injected into the planner's context when relevant.

Knowledge Base (RAG)

Embed your documents (PDF, DOCX, TXT, CSV, JSON) for semantic search. Documents are chunked (1000 chars, 200 overlap), embedded via z.ai API, and stored in a local vector store. The agent can search the knowledge base to answer questions about your documents.

8. Telegram Interface

8.1 Configuration

The Telegram interface is the primary channel for remote control. After creating your bot via @BotFather and getting the token, add it to the .env file. Important: set your user ID in the allowed_user_ids field in the Telegram configuration. This restricts bot usage to you alone, preventing anyone else from giving orders to your agent.

8.2 Slash Commands

Command	Description
/start /status /help	Agent status and help.
/screenshot	Instant screenshot sent to Telegram.
/pause /resume /cancel	Control the agent.
/memory	View memory state.
/files organize	Sort Downloads folder by file type.
/email unread	Read 5 latest unread emails with summary.
/email send to | subject | body	Send an email.
/calendar list	List 10 upcoming events.
/system info	Show system info (CPU, RAM, disk).
/browser open url	Open a URL in the browser.

8.3 Voice Messages

Send voice messages to the Telegram bot — the agent transcribes them via Whisper (API or local) and processes the transcript as a normal text request. The transcript is shown to you before processing. This enables hands-free operation while driving, walking, or cooking.

8.4 Proactive Notifications

The agent pushes notifications to your Telegram — it doesn't just respond. You'll receive: task started/completed/failed, calendar reminders (X minutes before events), new email alerts (from configured urgent senders), disk almost full (>90%), sustained high CPU (>90%), and custom alerts from any module.

9. Dashboard Web Interface

9.1 Overview

The dashboard is a Next.js 16 app with a cinematic command-center UI. It features glassmorphism cards, neon glow effects, a particle background, and a state orb that breathes and changes color with the agent's state. All panels update in real time via WebSocket.

9.2 Key Components

State Orb

Breathing/pulsing centerpiece that changes color (green=idle, amber=planning, cyan=executing) and rotation speed based on agent state.

Thinking Stream

Live ReAct trace with timeline dots, animated pulse on latest turn, typewriter 'thinking' cursor. Shows thought, action, observation, critique for each turn.

Module Grid

14 module tiles with per-module colors (screen=cyan, files=emerald, email=amber, etc.) and hover glow.

Command Palette

Cmd+K to submit tasks from anywhere in the dashboard.

Activity Heatmap

90-day GitHub-style grid with streak counter and success rate.

Cost Tracker

Total cost, API calls, tokens, per-model breakdown, period selector (today/week/month/all).

Audit Log

Live security trail with blocked-only filter.

Scheduled Tasks

CRUD for recurring tasks with inline create form.

Knowledge Base

Semantic search with score badges.

LLM Provider Switcher

List all 10 providers, set primary, test connections.

Prompt Templates

Browse and use 8 built-in + custom templates with variables.

Smart Suggestions

Predicted next actions as clickable chips under the task input.

Backup Panel

Create and restore full backups.

Bilingual EN/FR

Language toggle in header, auto-detected from browser locale.

10. Advanced Features

10.1 File Watcher

Trigger agent tasks when files change. Watch directories for created/modified/deleted/moved events with pattern matching and debounce. Example: 'When a new PDF appears in ~/Downloads, summarize it.' Rules are configurable from the dashboard.

10.2 Webhooks

Expose the agent via HTTP webhooks for external integrations. Each webhook has a unique secret URL, optional auth token, sync/async mode, and a template that maps incoming JSON to a task request. Use cases: GitHub PR review, Stripe payment alerts, Slack slash commands, IoT triggers.

10.3 Prompt Templates

8 built-in templates (Summarize PDF, Organize Folder, Email Reply, Meeting Prep, Code Review, Daily Summary, Translate, Research) plus unlimited custom templates. Templates support {variable} placeholders, categories, tags, and import/export for sharing.

10.4 Smart Suggestions

The agent predicts your next action based on task transition patterns and frequency analysis. Suggestions appear as clickable chips under the task input, adapting as you type.

10.5 Backup & Restore

Full backup of all agent data: memory, conversations, skills, templates, scheduled tasks, watch rules, webhooks, cost records, audit log, activity data, vector memories, knowledge base. Backups are ZIP files with metadata. Restore can be full or selective.

10.6 Cost Tracker

Tracks every API call's token usage and estimated cost (USD) with per-model pricing. Aggregations by period (today/week/month/all), by model, by role, and by task. Includes a trend chart and top tasks by cost. Records are cached for 30 seconds for fast dashboard updates.

10.7 Audit Log

Append-only security trail of every action. Records who triggered it (user ID, source), what action was attempted, when (ISO 8601), whether it was allowed or blocked (with reason), and the result (success/failure + error). Sensitive parameters (password, token, api_key) are automatically redacted. Essential for security investigations and compliance (GDPR, SOC2).

11. Security and Best Practices

11.1 Security Policy

Z.AGENT is configured by default in full autonomy mode: the agent can execute destructive actions without asking for confirmation. This mode is suitable for trusted use where you are the only one using the agent and you accept the risks. However, even in full autonomy, certain protections remain active at all times to prevent catastrophes.

Protected paths: ~/.ssh, ~/.aws, ~/.config/1password, /etc/passwd, /etc/shadow are never accessible in read, write, or delete mode. This list is configurable in the security.protected_paths section of the config file. Add any sensitive directory specific to your environment.

Blocked actions: certain actions are forbidden by default regardless of mode. These include format_disk, rm_rf_root, modify_system_files, shutdown_system, reboot_system. The agent will always refuse to execute them, even if you ask explicitly.

11.2 Best Practices

Restrict Telegram access

Configure allowed_user_ids with your Telegram ID. Without this, anyone knowing your bot name could send it orders.

Use app passwords

Never put your real email password in the config. Always use a dedicated, revocable app password.

Backup regularly

Use the Backup Panel to create regular backups. The agent can delete files — backups are your safety net.

Limit app whitelist

Only put apps the agent really needs to launch in system.allowed_apps. The shorter the list, the smaller the attack surface.

Monitor the audit log

Check the audit log regularly. If you see unexpected actions, pause the agent and investigate.

Test in dry-run

For sensitive tasks (file organization, deletions), use dry_run when available to see what would be done without actually doing it.

Update dependencies

Update Python and npm dependencies periodically to benefit from security fixes. Run pip-audit and bun audit.

12. Examples

12.1 Organize Downloads Folder

Send to the agent:

```
"Sort my Downloads folder by file type"
```

The agent will: (1) list the contents of ~/Downloads, (2) create subfolders by category (Images, Documents, Archives, Code, Videos, Audio), (3) move each file to the right subfolder, (4) send you a summary of the number of files moved per category. Files with unrecognized extensions stay at the root.

12.2 Read and Summarize Emails

```
"Read my 5 latest unread emails and give me a summary"
```

The agent will: (1) connect via IMAP to your inbox, (2) retrieve the 5 latest unread messages, (3) for each email extract sender, subject, and a summary of the body via GLM-4.6, (4) send everything to you via Telegram. You can then ask the agent to reply to any of them.

12.3 Research a Topic

```
"Research the topic: best practices for Python async. Find 3 sources and summarize."
```

The agent will: (1) use the web search module to find results, (2) read the top 3 pages, (3) extract key information from each, (4) synthesize a final answer with citations. This demonstrates the web.research action which combines search + read + synthesize.

12.4 Monitor a Website

```
"Open https://example.com/product, take a screenshot, and tell me the price."
```

The agent will: (1) launch Chromium via Playwright, (2) navigate to the URL, (3) wait for the page to load, (4) take a screenshot, (5) send the screenshot to GLM-4V with a prompt asking for the price, (6) return the answer. You can combine this with the scheduler to repeat hourly.

12.5 Voice Control

Send a voice message to your Telegram bot saying 'Take a screenshot and tell me what's on screen'. The agent will: (1) download the voice message, (2) transcribe it via Whisper, (3) show you the transcript, (4) execute the task, (5) send you the result. Completely hands-free.

13. Troubleshooting and FAQ

13.1 Agent doesn't respond on Telegram

Check that the Telegram token is correctly set in .env and that the agent is running (you should see 'Telegram interface started' in the logs). Then check that your user ID is in allowed_user_ids. If the bot is online but doesn't respond, check the agent logs — a z.ai API connection error is often the cause (expired key, exceeded quota, network problem). Try adding a fallback provider (e.g., OPENAI_API_KEY) so the agent can continue even if z.ai is down.

13.2 Agent clicks in the wrong place

The VLM module can sometimes mislocate UI elements. Increase the confidence threshold screen.click_confidence to 0.8 or 0.9 to require higher certainty. Also reduce screen.scale to 0.5 — lower resolution speeds up VLM processing but may reduce precision. For critical elements, use screen.find_and_click which automatically retries if the first attempt fails.

13.3 Email sending fails

The most common cause is using the real password instead of an app password. For Gmail, enable 2FA then generate an app password at myaccount.google.com/apppasswords. Also verify that imap_ssl and smtp_tls are set to true in the config. For Outlook, use imap-mail.outlook.com:993 and smtp-mail.outlook.com:587.

13.4 Dashboard doesn't connect to API

Check that the FastAPI is running on port 8765 (you should see 'Uvicorn running on http://127.0.0.1:8765' in the logs). If the API is on another machine, set NEXT_PUBLIC_AGENT_API. Check firewall rules — port 8765 must be open. Open the browser console to see WebSocket or fetch errors. CORS errors are resolved by adding your origin in dashboard.cors_origins in the config file.

13.5 High token consumption

Each task consumes tokens for planning (GLM-4.6) and potentially perception (GLM-4V). To reduce consumption: use slash commands for simple tasks (they bypass the planner), reduce max_actions_per_task to limit long plans, reduce screen.scale to reduce image sizes sent to VLM, use the Cost Tracker panel to monitor spending, and switch to cheaper providers like Groq or DeepSeek for the executor role.

Annex A — Full Action Reference

Complete list of all 88 actions available to the planner. Each action accepts the indicated parameters and returns a dictionary with at minimum a 'success' key (boolean).

Screen Control (10)

Action	Parameters
screen.click_element	description: str, button: str = 'left', clicks: int = 1
screen.click_xy	x: int, y: int, button: str = 'left', clicks: int = 1
screen.type_text	text: str, interval: float = 0.0
screen.press_key	key: str, presses: int = 1
screen.hotkey	keys: List[str]
screen.scroll	direction: str, amount: int = 3, x: int?, y: int?
screen.screenshot	description: str?
screen.wait	seconds: float = 1.0
screen.find_and_click	description: str, max_retries: int = 2
screen.drag	x1, y1, x2, y2: int, duration: float = 0.5

File Manager (10)

Action	Parameters
files.list	path: str?, pattern: str = '*'
files.move	sources: List[str], destination: str
files.copy	sources: List[str], destination: str
files.rename	path: str, new_name: str
files.delete	path: str, permanent: bool = false
files.organize	path: str?, dry_run: bool = false
files.search	path: str?, pattern: str, content_query: str?
files.read	path: str, max_size: int = 1MB
files.write	path: str, content: str, append: bool = false
files.create_dir	path: str

Email (6)

Action	Parameters
email.send	to, subject, body: str, cc?, bcc?, attachments?, html?
email.read_unread	folder: str = 'INBOX', limit: int = 10
email.search	query: str, folder: str = 'INBOX'
email.reply	message_id: str, body: str
email.mark_read	message_id: str
email.list_folders	—

Calendar (5)

Action	Parameters
calendar.list	days_ahead: int = 7
calendar.create	title, start: str, end?, description?, location?
calendar.delete	uid: str
calendar.search	query: str
calendar.remind	event_uid: str, minutes_before: int = 15

Browser (8)

Action	Parameters
browser.open	url: str, wait_until: str = 'domcontentloaded'
browser.click	selector: str, wait: bool = true
browser.fill	selector: str, value: str
browser.screenshot	full_page: bool = false
browser.extract	selector: str = 'body', attribute: str
browser.scroll	direction: str, amount: int = 500
browser.evaluate	script: str
browser.close	—

System (9)

Action	Parameters
system.launch_app	name: str, args: List[str]?
system.kill_app	name: str
system.list_processes	filter_name: str?, limit: int = 50
system.notification	title: str, message: str
system.clipboard_get	—
system.clipboard_set	content: str
system.open_path	path: str
system.system_info	—
system.run_command	command: str, cwd: str?, timeout: int = 60

Windows Control (25)

Action	Parameters
windows.powershell	command: str, timeout: int = 60, elevation: bool = false
windows.registry_read	hive, path, name: str
windows.registry_write	hive, path, name: str, value, reg_type?
windows.registry_delete	hive, path: str, name?
windows.service_list	filter_state: str?
windows.service_start / stop	name: str
windows.window_list	—
windows.window_focus / close / minimize / maximize	title? or hwnd?
windows.window_move	title? or hwnd?, x, y, width, height: int
windows.set_volume	level: int (0-100)
windows.set_brightness	level: int (0-100)

windows.set_wallpaper	path: str
windows.list_installed_apps	—
windows.uninstall_app	name: str
windows.list_wifi	—
windows.connect_wifi	ssid: str, password?
windows.event_log	log_name?, max_entries?, level?
windows.com_invoke	prog_id, method: str, args?
windows.taskbar_pin	app_path: str, pin: bool
windows.env_get / env_set	name: str, value? (set only)

Code Interpreter (4)

Action	Parameters
code.run_python	code: str, timeout: int?
code.evaluate	expression: str
code.list_files	—
code.read_file	name: str

Web Search (4)

Action	Parameters
web.search	query: str, num: int?
web.read_page	url: str
web.fetch	url: str, extract_text: bool?
web.research	topic: str, depth: int?

Voice Control (3)

Action	Parameters
voice.transcribe	audio_path: str, language: str?

voice.speak	text: str, voice: str?, speed: float?
voice.list_voices	—

Vision Stream (5)

Action	Parameters
vision.start_stream	mode: str?, fps: float?
vision.stop_stream	—
vision.get_status	—
vision.watch_for	description: str, timeout_s: int?
vision.wait_for_change	timeout_s: int?

Plugin Marketplace (7)

Action	Parameters
plugin.list	—
plugin.install_path	source_path: str, force: bool?
plugin.install_url	url: str, force: bool?
plugin.enable / disable	name: str
plugin.uninstall	name: str
plugin.info	name: str

MCP Client (5)

Action	Parameters
mcp.list_servers	—
mcp.list_tools	server_name: str?
mcp.call_tool	server_name, tool_name: str, arguments?
mcp.connect / disconnect	server_name: str

Knowledge Base (5)

Action	Parameters
`kb.add_document`	`file_path: str, name: str?`
`kb.search`	`query: str, top_k: int?`
`kb.list_documents`	—
`kb.delete_document`	`doc_id: str`
`kb.get_stats`	—

Slack (4)

Action	Parameters
`slack.send_message`	`text: str, channel: str?, blocks?`
`slack.list_channels`	—
`slack.send_file`	`file_path: str, channels?, title?`
`slack.list_messages`	`channel: str, limit: int?`